

Inhaltsverzeichnis

1 Einführung und Ziele	1
1.1 Fachliche Anforderungen.....	2
1.2 Qualitätsziele	2
1.3 Beteiligte	2
2 Randbedingungen	2
3 Kontextabgrenzung	2
3.1 Externe Schnittstellen	3
4 Lösungsstrategie	3
5 Bausteinsicht	3
5.1 Ebene 1: Das Gesamtsystem	3
5.2 Ebene 2: Der Order Service im Inneren.....	3
6 Laufzeitsicht.....	3
6.1 Anmeldung und Zugriff auf geschützte Bereiche	3
6.2 Kauf von Artikeln	3
7 Verteilungssicht.....	3
8 Querschnittliche Konzepte	3
8.1 Sicherheit.....	3
8.2 Kommunikation zwischen den Services	3
8.3 Datenhaltung.....	3
8.4 Fehlerbehandlung.....	3
9 Architekturentscheidungen	3
10 Qualitätsanforderungen	3
11 Risiken und technische Schulden	3
12 Fazit.....	3

1 Einführung und Ziele

1.1 Fachliche Anforderungen

1.2 Qualitätsziele

1.3 Beteiligte

Rolle	Erwartung an die Architektur
Projektgruppe (Entwicklung)	Klar abgegrenzte Zuständigkeiten, damit parallel an mehreren Services gearbeitet werden kann.
Dozent (Bewertung)	Nachvollziehbare Begründung des Architekturstils sowie eine lauffähige Demonstration.
Endnutzer (fachlich)	Ein funktionierender Kaufprozess mit verllässlicher Rückmeldung per E-Mail.

2 Randbedingungen

Randbedingung	Erläuterung
Organisatorischer Rahmen	Beispielanwendung zur Veranschaulung einer Softarchitektur und klarer Darstellung der wesentlichen Prinzipien. Gruppenprojekt im Rahmen des Moduls Softwarearchitektur
Freie Technologiewahl	Es gab keine Vorgabe für den Technologie-Stack. Diese Freiheit wurde bewusst genutzt, um die für Microservices typische Technologieoffenheit tatsächlich zu erproben statt sie nur zu behaupten.
Lokale Ausführbarkeit	Das gesamte System muss isoliert und auf einem normalen Computer startbar sein. Daraus folgt der Einsatz von Docker Compose anstelle einer Cloud-Umgebung.

3 Kontextabgrenzung

Das System tritt nach außen als eine einzige Anwendung auf. Als einzige und zentrale Zugang zum System erfolgt über den Browser. Daraus ergeben sich nach vom System nach außen lediglich zwei Schnittstellen. Die einzige Verbindung zu einem fremden System besteht dabei zum Mailserver, über den die Bestellbestätigungen versendet werden.

3.1 Externe Schnittstellen

Nachbar	Richtung	Inhalt und Protokoll
---------	----------	----------------------

Nutzer (Käufer und Verkäufer)	eingehend	Bedienung über den Browser: Anmeldung, Artikelverwaltung und Kauf über HTTP/REST.
SMTP-Server (Gmail)	ausgehend	Versand der Bestellbestätigung über SMTP mit STARTTLS auf Port 587.

4 Lösungsstrategie

Die Architektur des Systems folgt dem Stil der Microservicearchitektur. Dessen Stärken liegen in Isolation, Skalierbarkeit und Unabhängigkeit der einzelnen Komponenten. Dies bietet eine stabile Grundlage für ein komplexes System, wie in diesem Projekt. Aus dieser Grundsatzentscheidung leiten sich die folgenden zentralen Architekturentscheidungen ab. Die nachstehende Tabelle gliedert diese strukturiert auf

Entscheidung	Umsetzung im Projekt
Schnitt entlang fachlicher Grenzen	Fünf fachliche Services: Authentifizierung, Nutzerprofile, Artikelkatalog, Bestellungen und Benachrichtigungen.
Ein zentraler Eingang	Ein zentrales API Gateway nimmt alle Anfragen entgegen, prüft das Zugriffstoken und verteilt die Anfragen an den zuständigen Service.
Eine Datenbank je Service	Jeder Service besitzt eine eigene PostgreSQL-Instanz, auf die kein anderer Service zugreifen darf.
Kommunikationsart nach Bedarf	Innerhalb des Kaufvorgangs wird synchron über REST kommuniziert. Die Benachrichtigung läuft hingegen asynchron über einen MQTT-Broker.
Skalierung dort, wo Last entsteht	Der Produktservice läuft in drei Instanzen hinter einem Nginx-Lastverteiler, alle übrigen Services einfach.
Bewusst gemischter Stack	Java mit Spring Boot für die datenintensiven Services, Python für Authentifizierung und Mailversand, Astro für die Oberfläche.

5 Bausteinsicht

5.1 Ebene 1: Das Gesamtsystem

Das System besteht aus sieben eigenständig ausgelieferten Bausteinen sowie einem Nachrichtenbroker. Die folgende Abbildung zeigt sie zusammen mit ihren Datenbanken und den Kommunikationsbeziehungen.

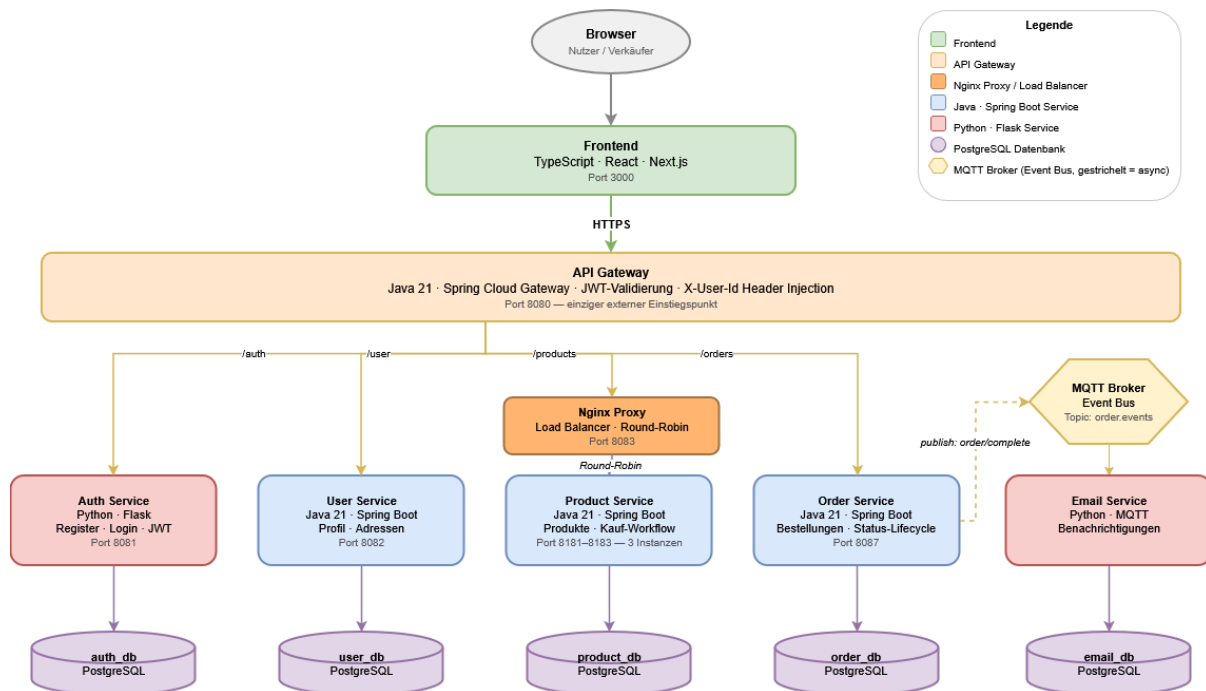


Abbildung 2: Bausteinsicht Services, Datenbanken und Nachrichtenbroker

Die Verantwortlichkeiten der einzelnen Bausteine sind wie folgt abgegrenzt:

Baustein	Verantwortung	Technologie	Eigene Datenbank
Frontend	Oberfläche für Produkte, Warenkorb und Nutzerkonto. Es dient als Darstellungsschicht und leitet alle Aufrufe an das Gateway weiter.	Astro 5	nein
API Gateway	Es dient als zentraler Eingang von außen und authentifiziert und Anfragen und leitet auf Basis des Pfades an den zuständigen Service weiter.	Spring Cloud Gateway, Java 21	nein
Auth Service	Registrierung, Anmeldung und Ausstellung der Zugriffstoken.	Python, Flask	auth-service-db
User Service	Nutzerprofile und Lieferadressen.	Spring Boot	user-service-db
Product Service	Produktkatalog, Statusverwaltung der Artikel und Anstoßen des Kaufvorgangs.	Spring Boot, 3 Instanzen	product-db
Order Service	Bestellungen mit ihren Positionen, Bestellstatus und Auslösen der Benachrichtigung.	Spring Boot	order-service-db
Email Service	Empfängt Bestellereignisse, füllt die Mailvorlage und versendet die Bestätigung.	Python, paho-mqtt	email-service-db
MQTT-Broker	Vermittelt Ereignisse zwischen Order Service und Email Service.	Eclipse Mosquitto 2	nein

5.2 Ebene 2: Der Order Service im Inneren

Stellvertretend für die Java-Services wird der Order Service genauer betrachtet. Sein innerer Aufbau folgt einer klassischen Schichtung: Der Controller nimmt Anfragen entgegen, die Serviceklasse enthält die Geschäftslogik und das Repository kapselt den Datenbankzugriff. Damit der Service möglichst

Technologie unabhängig bleibt erfolgen die Aufrufe über eine austauschbare Clientklasse, welche die REST-Aufrufe oder Veröffentlichung von MQTT-Events übernimmt.

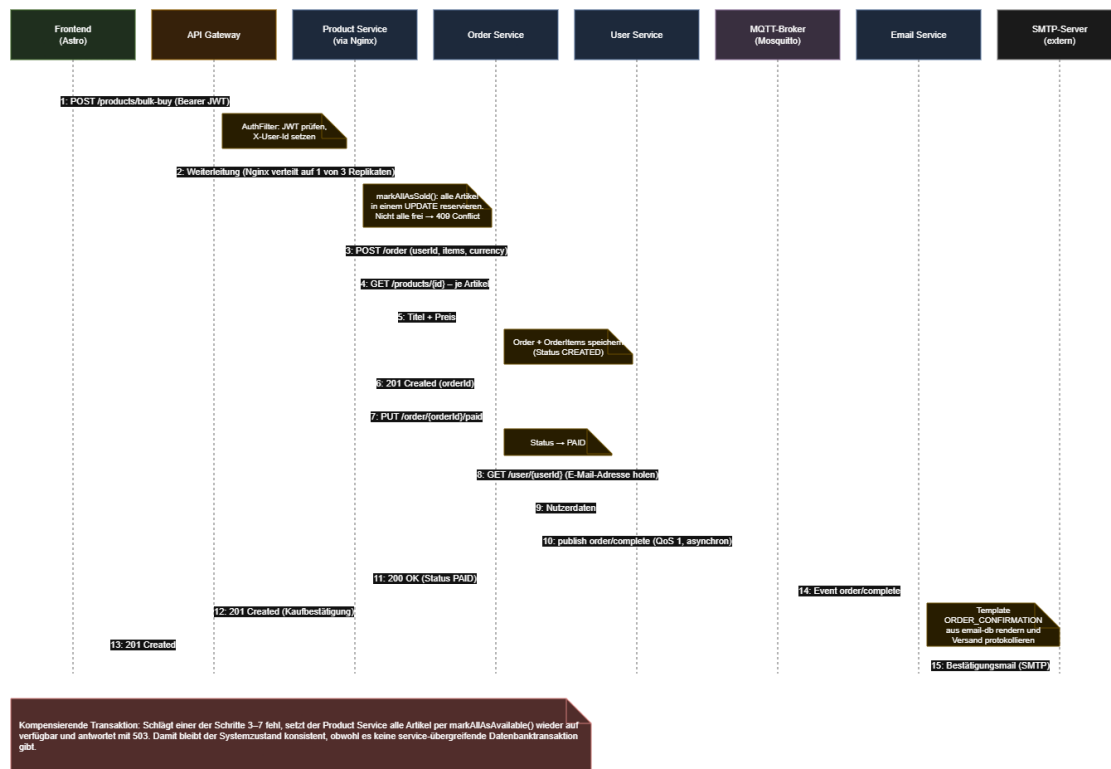


Abbildung 3: Bausteinsicht Ebene 2: Order Service als Whitebox

6 Laufzeitsicht

6.1 Anmeldung und Zugriff auf geschützte Bereiche

Teile der Anwendung bedürfen eines geschützten Bereichs bzw. einer Authentifizierung, sodass ein nicht angemeldeter Nutzer keinen Zugriff darauf hat. Zudem ist es erforderlich, dass manche Ressourcen nur von Nutzern mit ausreichenden Zugriffsrechten eingesehen oder manipuliert werden dürfen. Hierbei kommt der JSON Web Token (JWT) zum Einsatz. Nach erfolgreicher Anmeldung eines Nutzers mittels Nutzernamen oder E-Mail und Passwort wird ein 60 Minuten gültiger Token erzeugt. Dieser Token dient als eindeutige Authentifizierung für den Nutzer und wird bei jedem Aufruf eines geschützten Endpunkts verifiziert und validiert. Die entsprechende Nutzer-ID wird extrahiert und im Header als Kontext für nachfolgende Services weitergegeben.

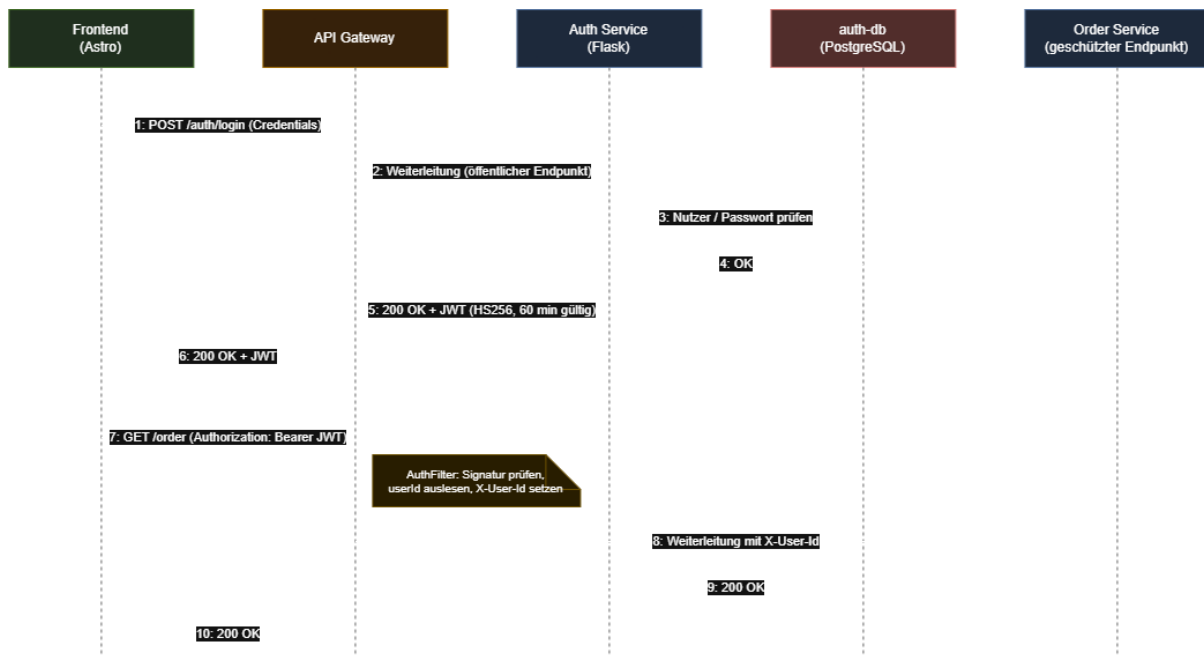


Abbildung 4: Laufzeitsicht: Anmeldung und anschließender Zugriff mit Token

6.2 Kauf von Artikeln

Der Kaufvorgang ist das aufschlussreichste Szenario, weil in ihm alle Kommunikationsarten des Systems zusammentreffen. Ausgelöst wird er durch einen einzigen Aufruf des Frontends, der sämtliche Artikel des Warenkorbs auf einmal übergibt. Der Product Service reserviert zunächst alle betroffenen Artikel in einer einzigen Datenbankanweisung. Gelingt das nicht für jeden Artikel, war mindestens einer bereits verkauft, und der gesamte Vorgang wird mit einem Konfliktfehler abgewiesen. Erst danach legt der Product Service über den Order Service die Bestellung an und markiert sie als bezahlt.

Genau dieser Zeitpunkt löst die Benachrichtigung aus: Der Order Service beschafft die E-Mail-Adresse beim User Service und veröffentlicht ein Ereignis auf dem MQTT-Broker. Die Antwort an den Nutzer wird abgeschickt, ohne den Versand der E-Mail abzuwarten. Der Email Service greift das Ereignis unabhängig davon auf, füllt die hinterlegte Mailvorlage und versendet die Bestätigung.

[Abbildung 5 hier einfügen]

Quelle: *diagramme/03-sequenz-produkt-kaufen.drawio*
 In Draw.io öffnen und als PNG (Zoom 200 %) exportieren, dann hier einfügen.

Abbildung 5: Laufzeitsicht: Kauf mehrerer Artikel inklusive Benachrichtigung

Umgang mit Fehlern. Da eine Bestellung mehrere Services und mehrere Datenbanken berührt, kann sie nicht durch eine gemeinsame Datenbanktransaktion abgesichert werden. Stattdessen kommt eine kompensierende Transaktion zum Einsatz: Scheitert das Anlegen der Bestellung, setzt der Product

Service alle zuvor reservierten Artikel wieder auf „verfügbar“ und meldet dem Aufrufer, dass der Kauf zurückgenommen wurde. Der Datenbestand bleibt damit auch im Fehlerfall widerspruchsfrei.

7 Verteilungssicht

Alle Bausteine des Systems laufen als Docker-Container. Dabei folgt der Zuschnitt der einzelnen Container der Struktur der Services. Jeder Service ist über eine Compose-Datei definiert und kann somit unabhängig und eigenständig deployed werden. Eine übergeordnete Compose-Datei bindet diese einzelnen Dateien lediglich ein, ohne selbst Konfigurationsdetails zu enthalten. Dadurch lässt sich jeder Service auch isoliert starten, testen oder aktualisieren, ohne dass an der zentralen Datei etwas angepasst werden muss.

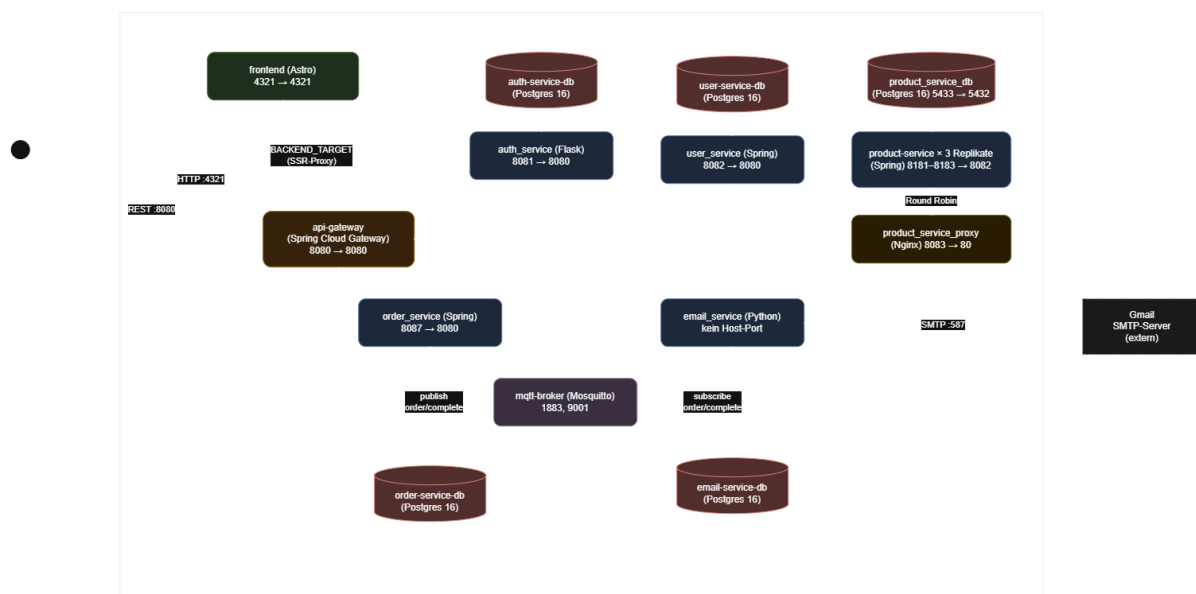


Abbildung 6: Verteilungssicht: Container, Ports und Netzwerkbeziehungen

Innerhalb des Docker-Netzwerks sprechen sich die Container über ihre Servicenamen an, sodass keine IP-Adressen konfiguriert werden müssen. Nach außen sind nur die in der folgenden Tabelle genannten Ports sichtbar.

Container	Port außen	Port innen	Anmerkung
frontend	4321	4321	Oberfläche im Browser
api-gateway	8080	8080	einziger Eingang für alle Schnittstellenaufrufe
auth_service	8081	8080	
user_service	8082	8080	
product-service_proxy	8083	80	Nginx als Lastverteiler
product-service	8181–8183	8082	drei Instanzen
order_service	8087	8080	

email_service	–	–	nur über MQTT erreichbar, kein offener Port
mqtt-broker	1883, 9001	1883, 9001	Eclipse Mosquitto
Datenbanken	nur product-db: 5433	5432	je Service eine PostgreSQL- 16-Instanz

8 Querschnittliche Konzepte

8.1 Sicherheit

Die Zugriffsprüfung ist vollständig im Gateway gebündelt. Ein Filter unterscheidet dort zwischen offenen und geschützten Pfaden: Anmeldung, Registrierung sowie das reine Ansehen von Artikeln sind ohne Token erreichbar, alle übrigen Aufrufe erfordern ein gültiges Token. Ist es gültig, ergänzt das Gateway die Nutzerkennung als Kopfeintrag; fehlt es oder ist es ungültig, wird die Anfrage sofort abgewiesen und erreicht den Zielservice gar nicht erst. Dieses Vorgehen entspricht dem Muster des zentralen Zugangspunkts und verhindert, dass die Prüflogik in jedem Service dupliziert wird.

8.2 Kommunikation zwischen den Services

Das System nutzt bewusst zwei Kommunikationsarten. Synchrone Aufrufe über REST kommen immer dann zum Einsatz, wenn der Aufrufer die Antwort zwingend benötigt, um weiterarbeiten zu können – etwa wenn der Order Service Titel und Preis eines Artikels braucht. Asynchrone Nachrichten über den MQTT-Broker werden dort verwendet, wo das Ergebnis für den laufenden Vorgang unerheblich ist. Der Versand der Bestellbestätigung ist genau so ein Fall: Der Kauf ist bereits abgeschlossen, wenn das Ereignis veröffentlicht wird.

Der Gewinn dieser Trennung zeigt sich beim Ausfall: Ist der Email Service nicht verfügbar, bleibt die Nachricht im Broker liegen und der Kaufvorgang läuft unbeeinträchtigt weiter. Bei einem synchronen Aufruf hätte derselbe Ausfall den Kauf zum Scheitern gebracht.

8.3 Datenhaltung

Jeder Service besitzt seine eigene Datenbank, und kein Service greift auf die Datenbank eines anderen zu. Diese strikte Trennung ist die Voraussetzung dafür, dass ein Service sein Datenmodell ändern kann, ohne andere zu gefährden. Sie hat allerdings ihren Preis: Da Verknüpfungen über Servicegrenzen hinweg nicht möglich sind, werden fremde Datensätze nur über ihre Kennung referenziert und bei Bedarf per Aufruf nachgeladen. Zusammenhängende Auswertungen über mehrere Services erfordern damit mehrere Aufrufe statt einer einzigen Datenbankabfrage.

8.4 Fehlerbehandlung

Die Java-Services verwenden eine gemeinsame Struktur für Fehlerantworten, sodass das Frontend unabhängig vom aufgerufenen Service dasselbe Format erhält. Fachliche Fehlerfälle werden dabei auf sprechende Statuscodes abgebildet: Ein bereits verkaufter Artikel führt zu einem Konflikt, ein nicht erreichbarer Order Service zu einer Meldung über die vorübergehende Nichtverfügbarkeit.

9 Architekturentscheidungen

Nr.	Entscheidung	Begründung	Konsequenz
AE 1	Microservices statt Monolith	Der Stil sollte im Modul praktisch erfahrbar werden, und die Domäne lässt sich klar in fachliche Bereiche schneiden.	Höherer Aufwand für Betrieb und Kommunikation, dafür unabhängige Entwicklung und Auslieferung sowie Fehlerisolation.
AE 2	Zentrales API Gateway mit Tokenprüfung	Die Zugriffsprüfung sollte nicht in fünf Services parallel gepflegt werden.	Die Services werden einfacher, das Gateway wird jedoch zur zentralen Stelle, deren Ausfall das gesamte System betrifft.
AE 3	Eine Datenbank je Service	Eine gemeinsame Datenbank hätte alle Services über das Schema aneinander gekoppelt und die unabhängige Auslieferung verhindert.	Volle Hoheit über das eigene Datenmodell, dafür keine service-übergreifenden Verknüpfungen und Transaktionen.
AE 4	MQTT für Benachrichtigungen	Der Mailversand ist für den Abschluss des Kaufs nicht erforderlich und sollte ihn daher nicht blockieren können.	Der Kauf bleibt bei Ausfall des Email Service möglich; der Versandzeitpunkt ist dafür nicht mehr garantiert.
AE 5	Gemischter Technologie-Stack	Die Technologiefreiheit des Stils sollte tatsächlich erprobt und nicht nur behauptet werden.	Belegt die Unabhängigkeit der Services, erhöht aber den Einarbeitungsaufwand im Team.
AE 6	Drei Instanzen des Product Service	Der Katalog wird deutlich häufiger gelesen als bestellt wird.	Gezielte Skalierung genau des belasteten Bausteins; setzt voraus, dass der Service zustandslos bleibt.
AE 7	Kompensation statt verteilter Transaktion	Eine Transaktion über mehrere Datenbanken hinweg ist im gewählten Stil nicht verfügbar.	Der Datenbestand bleibt auch im Fehlerfall konsistent, die Rücknahmelogik muss jedoch von Hand programmiert werden.

10 Qualitätsanforderungen

Die Qualitätsziele aus Kapitel 1 werden hier zu Szenarien verfeinert, die sich in der Vorführung tatsächlich zeigen lassen.

Ziel	Szenario	Erwartete Reaktion des Systems
Loose Kopplung	Der Email Service wird gestoppt, anschließend wird ein Artikel gekauft.	Der Kauf wird erfolgreich abgeschlossen. Das Ereignis bleibt im Broker und die Bestätigung wird

		versendet, sobald der Service wieder läuft.
Skalierbarkeit	Mehrere Katalogabfragen treffen gleichzeitig ein.	Der Lastverteiler verteilt die Anfragen reihum auf die drei Instanzen des Product Service.
Unabhängige Auslieferung	Der Order Service wird neu gebaut und neu gestartet.	Katalog und Anmeldung bleiben währenddessen bedienbar; nur Bestellungen sind kurzzeitig nicht möglich.
Sicherheit	Ein geschützter Aufruf wird ohne oder mit ungültigem Token abgesetzt.	Das Gateway antwortet mit „nicht autorisiert“; der Zielservice wird nicht erreicht.
Konsistenz	Zwei Käufer versuchen gleichzeitig, denselben Artikel zu kaufen.	Nur ein Kauf gelingt. Der zweite erhält eine Konfliktmeldung, ohne dass eine unvollständige Bestellung zurückbleibt.

11 Risiken und technische Schulden

Punkt	Bewertung und möglicher Umgang
Gateway als zentrale Schwachstelle	Fällt das Gateway aus, ist das System von außen nicht mehr erreichbar. In einer produktiven Umgebung müsste es mehrfach betrieben werden.
Keine zentrale Protokollierung	Ein Kaufvorgang erzeugt Einträge in mehreren Services. Ohne zusammengeführte Logs und eine durchgehende Vorgangskennung ist die Fehlersuche mühsam.
Keine Dienstsuche	Adressen der Services stehen fest in der Konfiguration. Solange die Anzahl der Services klein bleibt, ist das vertretbar.
Zugangsdaten im Klartext	Signaturschlüssel und Mailzugangsdaten liegen unverschlüsselt in den Compose-Dateien. Für den Produktivbetrieb müssten sie ausgelagert werden; die verwendeten Daten sind vor einer Veröffentlichung des Projektarchivs zu ersetzen.
Wechselseitige Abhängigkeit	Product Service und Order Service rufen sich gegenseitig auf. Das funktioniert, erschwert aber die getrennte Betrachtung beider Services.
Ruhende Bausteine	Zahlungs- und Bilddienst sind angelegt, aber nicht in Betrieb. Sie sind für den Nachweis des Architekturstils nicht erforderlich.

12 Fazit

Der Aufbau der Anwendung hat die von Richards und Ford beschriebenen Eigenschaften des Stils in beide Richtungen bestätigt. Positiv fiel vor allem auf, wie unabhängig sich die Services entwickeln ließen: Weil jeder Baustein seine eigene Datenbank besitzt und nur über Schnittstellen erreichbar ist, konnten mehrere Personen gleichzeitig arbeiten, ohne sich gegenseitig zu blockieren. Ebenso deutlich

zeigte sich der Nutzen der Fehlerisolation: Der Ausfall des Email Service beeinträchtigt den Kaufvorgang nicht, weil beide über einen Broker entkoppelt sind.

Ebenso deutlich wurden die Nachteile. Ein fachlich einfacher Kaufvorgang erstreckt sich über vier Services und erfordert eine von Hand geschriebene Rücknahmelogik, die in einem Monolithen eine einzige Datenbanktransaktion gewesen wäre. Auch der Betriebsaufwand ist spürbar: Ein vollständiger Start umfasst mehr als ein Dutzend Container, und die Fehlersuche erfordert den Blick in mehrere Protokolle gleichzeitig.

Für die Frage, wann sich der Stil lohnt, ergibt sich daraus ein klares Bild. Microservices sind sinnvoll, wenn mehrere Teams unabhängig voneinander ausliefern müssen, wenn einzelne Systemteile deutlich unterschiedlich belastet werden oder wenn der Ausfall eines Bereichs die übrigen nicht erfassen darf. Für eine Anwendung mit überschaubarem Funktionsumfang und einem kleinen Team überwiegen dagegen die Kosten. Gemessen an rein fachlichen Maßstäben wäre für das hier umgesetzte Beispiel ein modularer Monolith die wirtschaftlichere Wahl gewesen – als Lernumgebung für die Eigenschaften verteilter Architekturen hat sich der gewählte Stil hingegen als sehr geeignet erwiesen.